

GramSetu AI - Software Architecture Specification

This document details the architectural layout, modules separation, global contexts, hooks pipelines, and data flow of GramSetu AI.

1. Overall System Architecture

GramSetu AI utilizes a local-first, zero-cloud architecture. It runs in-browser utilizing client hardware acceleration.

```
graph TB
    subgraph Client [Client Browser Environment]
        subgraph View [View Layer - React 19]
            Shell[AppShell]
            Dash[Dashboard View]
            AuditUI[SwachhAudit UI]
            JalUI[JalDrishti UI]
            LipiUI[GramLipi UI]
            SankalpUI[SwachhSankalp UI]
        end

        subgraph Core [Global Contexts & States]
            Theme[ThemeContext]
            AppState[AppContext]
            AuditState[AuditContext]
        end

        subgraph Engines [Local Computation Engines]
            ORT[ONNX Runtime Web]
            Tess[Tesseract.js OCR Worker]
            CV[Canvas Preprocessing]
        end

        subgraph IO [Local Data Store]
            LS[(LocalStorage API)]
        end
    end

    subgraph Desktop [Local Localhost Host]
        Ollama[Ollama Server - llama3.2:3b]
    end

    subgraph Remote [Remote Web Server]
        Static[Static CDN / Vercel host]
    end

    Static -- Static HTML/JS/ONNX Weights --> Client
    Shell --> Core
    AuditUI --> AuditState
    AuditUI --> ORT
    JalUI -- Pad Extraction --> CV
    LipiUI --> Tess
    LipiUI -- Simplification --> Ollama
    SankalpUI --> LS
    Core --> LS
```

2. Module Sandboxing & Interactions

Each module inside the **app/** directory (such as **swachh-audit**, **jaldrishti**, **gramlipi**, and **swachhsankalp**) is structured as an isolated sandbox containing its own components, services, and hooks. Shared layouts and core utility functions reside at the root.

```
graph TD
    subgraph Root_Layout [Root Workspace]
        Components[Shared Components - components/ui, components/layout]
        Contexts[Shared Contexts - contexts/ThemeContext, contexts/AppContext]
        Utils[Shared Utilities - utils/cn, utils/generateId]
        Constants[Shared Constants - constants/routes]
    end

    subgraph Module_Sandboxes [Module Boundaries]
        subgraph SwachhAudit [app/swachh-audit]
            AuditServices[services/detection]
            AuditHooks[hooks/useDetection]
            AuditComponents[components/DetectionPreview]
        end

        subgraph JalDrishti [app/jaldrishti]
            JalServices[services/imageProcessor]
            JalHooks[hooks/useStripAnalysis]
            JalComponents[components/WaterReportCard]
        end

        subgraph GramLipi [app/gramlipi]
            LipiServices[services/ocrService]
            LipiHooks[hooks/useDocumentOcr]
            LipiComponents[components/ocr-preview-card]
        end

        subgraph SwachhSankalp [app/swachhsankalp]
            SankalpServices[services/storage]
            SankalpHooks[hooks/useSwachhData]
            SankalpComponents[components/WardLeaderboard]
        end
    end

    SwachhAudit --> Components
    SwachhAudit --> Utils
    SwachhAudit --> Contexts

    JalDrishti --> Components
    JalDrishti --> Utils

    GramLipi --> Components
    GramLipi --> Utils

    SwachhSankalp --> Components
    SwachhSankalp --> Utils
    SwachhSankalp --> Constants
```

3. Data Flow & Local State Synchronization

Data persistence uses browser **localStorage**. Changes in UI components update the local React state, which is synchronously written to localStorage via hook wrappers.

```
graph TD
    UI[User Toggle Checklist / Join Campaign]
    UI --> StateUpdate[useSwachhData Hook State Updates]
    UI --> SaveStorage[storage.ts wrapper writes to localStorage]
    UI --> EventCheck[Trigger checkAchievements callback]
```

```
--> Compare[Verify rules: tasks completed / campaigns joined]
--> Unlock[Unlock Achievements Badge]
--> SyncState[Set achievements state]
--> Redraw[Update WardLeaderboard & score circle in UI]
```

4. AI Inference Execution Flow (SwachhAudit)

The pipeline for processing garbage audits starts at camera capture, flows through scale/normalization canvas layers, runs through on-device models, and filters outputs using Non-Maximum Suppression (NMS).

```
graph TD
    Capture[Camera Snapshot / Data URL]
    --> Canvas[Canvas Preprocessor - imagePreprocessor.ts]
    --> Scale[Resize to 640x640 & Reorder RGBA to Planar RGB]
    --> Normal[Normalize to 0.0 - 1.0 Float32 Array]
    --> ORTSession[onnxruntime-web WASM Execution]
    --> OutTensor[Output Tensor - 1, 84, 8400]
    --> NMS[postProcessor.ts - Filter boxes by score >= 0.25]
    --> Deduplicate[Remove overlaps with IoU >= 0.45]
    --> ComputeScore[cleanliness.ts - Calculate Cleanliness Index]
    --> Report[Compile SwachhAuditReport payload]
    --> History[Store report in local history]
```

5. UI Component Hierarchy

The Root Layout renders the shell (**AppShell**) which houses navigation links, routing indicators, and the main page router.

```
graph TD
    RootLayout[Root Layout - layout.tsx]
    --> AppShell[AppShell.tsx]

    AppShell --> TopNavbar[TopNavbar.tsx]
    AppShell --> Sidebar[Sidebar.tsx]
    AppShell --> MobileNavigation[MobileNavigation.tsx]
    AppShell --> MainContent[Main Content Router]

    TopNavbar --> Breadcrumb[Breadcrumb.tsx]
    TopNavbar --> ThemeToggle[ThemeToggle / Profile]

    MainContent --> Dashboard[Dashboard - app/dashboard/page.tsx]
    MainContent --> SwachhAuditPage[SwachhAudit - app/swachh-audit/page.tsx]
    MainContent --> JalDrishtiPage[JalDrishti - app/jaldrishti/page.tsx]
    MainContent --> GramLipiPage[GramLipi - app/gramlipi/page.tsx]
    MainContent --> SwachhSankalpPage[SwachhSankalp - app/swachhsankalp/page.tsx]
```

6. Folder Dependency Flow

Dependencies flow from modular directories to root shared layers. Modules do not import from other modules.

```
graph TD
    AppFolder[app/ module folders]
    --> RootComponents[components/ shared UI components]

    AppFolder --> RootUtils[utils/ shared utilities]
    AppFolder --> RootHooks[hooks/ shared localStorage/mobile hooks]
```

AppFolder --> RootContexts[contexts/ global state contexts]

RootComponents --> RootUtils

RootHooks --> RootUtils

RootContexts --> RootHooks